CS193

Homework - Week 7

Introduction

This week you'll write a real, non-trivial bash script that works with **live data**. Your script builds a small **conditions dashboard** for **both Purdue campuses** — West Lafayette and Indianapolis. For each one it fetches the current weather and air quality from two free web APIs, reads the **JSON**, reports today's **temperature pattern** (high, low, and how many hours are "hot"), and logs every reading to a CSV so the data builds up over time.

You'll pull together most of the term's bash toolkit on genuine data: a reusable `function`, `for` loops over an array of locations *and* over hourly forecast values, variables and `$( )`, two `if`/`elif`/`else` decision ladders, error handling with exit codes, and saving output with `>>`. To read the JSON you'll use `jq`, the standard command-line JSON tool — on single values **and** on arrays.

We give you a working template with the campuses, URLs, function skeletons, and a `jq` guide already in place. You fill in the `TODO` sections. Every API is **free and needs no account or API key**.

Getting Started: The Template

Open the `hw7-conditions` folder and fill in the `TODO` sections.

- `conditions.sh` — the script you'll write. Look for `# TODO` comments; each names the lecture slide with the matching syntax. The top of the file has a `jq` guide covering single values **and** arrays.
- `README.md` — setup, run instructions, and example output.
- `.vscode/` — optional editor settings; safe to ignore if your editor doesn't use them.
- `conditions_report.txt` and `conditions_log.csv` — created by your script when it runs (a readable report and a growing CSV history).

One-Time Setup: Install `jq`

`jq` reads JSON from the command line. Install it once:

```
macOS:          brew install jq
Ubuntu/WSL:     sudo apt install jq
Windows:        choco install jq      (or winget install jqlang.jq)
Purdue "data":  already installed - just SSH in and go
```

Confirm with `jq --version`. New to `jq`? Here's the whole guide you need:

```
echo "$json" | jq '.current.temperature_2m'        # one value
echo "$json" | jq '.hourly.temperature_2m | max'   # array -> highest
echo "$json" | jq '.hourly.temperature_2m | min'   # array -> lowest
echo "$json" | jq -r '.hourly.temperature_2m[0:6][]'   # first 6, raw
```

```
# count array items greater than a number N:
echo "$json" | jq --argjson t 75 \
    '[.hourly.temperature_2m[] | select(. > $t)] | length'
```

How to Run Your Script

Open It and Run

1. Open the `hw7-conditions` folder in whatever code editor you prefer, or just open a terminal and `cd` into it — **any IDE or the terminal works**.

2. Make it executable (one time), then run it. You can pass a "hot hour" cutoff, or let it default to 75:

```
$ chmod +x conditions.sh
$ ./conditions.sh 80
=========================================================
 Purdue Conditions Dashboard - Sat Aug  2 14:10:00 EDT 2026
 (hot-hour cutoff: 80 F)
=========================================================
----- West Lafayette -----
Now:         81.5 F (Hot), humidity 58%
Air:         AQI 42 (Good), PM2.5 9.1
Today:       high 86.0 F / low 64.0 F, 8 hour(s) above 80 F
Next 6 hrs:
   64.0 F
   ...
----- Indianapolis -----
   ...
```

3. Numbers reflect the *live* weather. Run it again and open `conditions_log.csv` to watch the history grow. You need an internet connection for the fetch.

What to Fill In

Work through the `TODO` comments in `conditions.sh` in order:

1. **Finish `get_json`.** Make the helper `return 1` when the fetch fails (`$?` is not 0) **or** the response is empty (`-z`).

2. **Fetch the air feed.** Inside `report_for`, grab the air-quality URL into `air` using `get_json`, failing with a message if it doesn't come back.

3. **Read the scalars.** Temperature is done for you. Pull `humidity`, `aqi`, and `pm25` with the right `.current.*` `jq` paths.

4. **Summarize the day (arrays).** Use `jq` on the hourly array to set `high` (`max`), `low` (`min`), and `hot` (count of hours above `$THRESHOLD` — see the `jq` guide).

5. **Categorize.** Two `if/elif/else` ladders: AQI (0–50 Good, 51–100 Moderate, 101+ Unhealthy) and temperature (<40 Cold, <60 Cool, <80 Mild, else Hot).

6. **Print the next 6 hours.** A `for` loop over the first six hourly temps, echoing each one indented.

7. **Log to CSV.** Append one row — `date,name,temp,humidity,aqi,pm25` — to `$CSV` with `>>`.

8. **Loop over both campuses.** In `main`, `for` each entry in `LOCATIONS`, split it on `;` into `name/lat/lon` and call `report_for`.

Optional stretch (not required for full credit): add a third location, or print a one-line "best campus to be outside right now" verdict by comparing the two AQI values.

How to Submit

1. Run your script at least twice so `conditions_log.csv` has several rows.
2. Zip your `hw7-conditions` folder, with your finished `conditions.sh`, `conditions_report.txt`, and `conditions_log.csv` included:
   - **Windows:** Right-click the `hw7-conditions` folder → **Send to** → **Compressed (zipped) folder**.
   - **Mac:** Right-click the `hw7-conditions` folder → **Compress "hw7-conditions"**.
3. Go to Ed Discussion and open the **Week 7** submission thread.
4. Create a new post, attach your zipped folder, and submit.

Checklist Before You're Done

- `get_json` returns non-zero on an empty or failed fetch - 10%
- The air feed is fetched with a failure guard - 10%
- Humidity, AQI, and PM2.5 are read with correct `jq` paths - 15%
- Today's high, low, and hot-hour count come from `jq` on the hourly array - 20%
- AQI **and** temperature are each categorized with `if/elif/else` - 15%
- The next 6 hours are printed with a `for` loop - 10%
- Each reading is appended to `conditions_log.csv` - 10%
- `main` loops over **both** campuses (name/lat/lon split from the array) - 10%

Grading

This assignment is graded on completion of the checklist above. Each item is worth the percentage listed next to it, and the items add up to 100% of your grade on this assignment.